

ZAiV User Guide

On Raspberry-PI

목차

I	개요	6
1.	문서 목적	6
2.	History	7
II	실행 준비	8
1.	구성	8
1.1	환경 구성	8
1.2	파일 준비	8
2.	환경 구성	9
2.1	ZAiV IP 설정하기	9
2.2	종속성 설치	13
3.	실행	15
3.1	기본 예제	15
3.2	ROS 예제	16
III	모델 재학습	21
1.	구성	21
1.1	환경구성	21
2.	가상머신	21
2.1	설치	21
2.2	가상 환경 생성	22
3.	Yolov5 재학습 환경 구성	26
3.1	아나콘다 설치	26
3.2	Yolov5 설치 / 재학습	28
4.	DFC 환경 구성	32
4.1	DFC 설치	32
4.2	Data Flow Compiler 실행하기	34
5.	기타사항	39
5.1	예외	39

5.2 TroubleShooting 39

그림 목차

[그림 I-1] 시스템 구조	6
[그림 I-2] ZAI V EDGE BOARD	6
[그림 II-1] 환경 구성	8
[그림 II-2] RASPBERRY PI NETWORK INTERFACE	9
[그림 II-3] 카메라 번호	16
[그림 III-1] 환경구성	21
[그림 III-2] 설치 마법사 1	21
[그림 III-3] 설치 마법사 2	22
[그림 III-4] 가상 환경 생성 1	22
[그림 III-5] 가상 환경 생성 2	23
[그림 III-6] 가상 환경 생성 3	23
[그림 III-7] 가상 환경 생성 4	24
[그림 III-8] 가상 환경 생성 5	24
[그림 III-9] 가상 환경 생성 6	25
[그림 III-10] 라이선스 내용 확인	26
[그림 III-11] 저장 경로 지정	26
[그림 III-12] ROOT 계정 CONDA INIT 유무	27
[그림 III-13] YOLOV5 JUPYTER NOTEBOOK	27
[그림 III-14] GIT을 이용한 YOLOV5 설치	28
[그림 III-15] 데이터 셋 저장 공간	28
[그림 III-16] 이미지 라벨링 툴	29
[그림 III-17] DATA.YAML	29
[그림 III-18] 모델 학습	30
[그림 III-19] 모델 학습 결과	30
[그림 III-20] 모델 결과 테스트	30
[그림 III-21] YOLOV5 모델을 ONNX로 내보내기	31
[그림 III-22] HAILO -H 결과	33
[그림 III-23] ZAI V 폴더 이동	34
[그림 III-24] ONNX 모델 파싱	35
[그림 III-25] START NODE [그림 III-26] ENDNODE	35
[그림 III-26] 파싱 결과 저장 / 시각화	36
[그림 III-27] 파싱 결과 PROFILER 코드	36
[그림 III-28] 캘리브레이션 데이터 읽어오기	36
[그림 III-29] 이미지 전처리 함수 생성	36
[그림 III-30] 이미지 불러오기 & 전처리	37
[그림 III-31] 데이터셋 저장 & 확인하기	37

[그림 III-32] 모델 스크립트 및 최적화37

[그림 III-33] 최적화 결과 확인.....38

[그림 III-34] HEF 컴파일 코드.....38

I 개요

1. 문서 목적

본 문서는 RaspberryPi4 기반으로 Withus의 ZAiV Edge 보드를 통해 YOLOv5 모델을 구동시키기 위한 가이드입니다.



[그림 I-1] 시스템 구조



[그림 I-2] ZAiV Edge Board

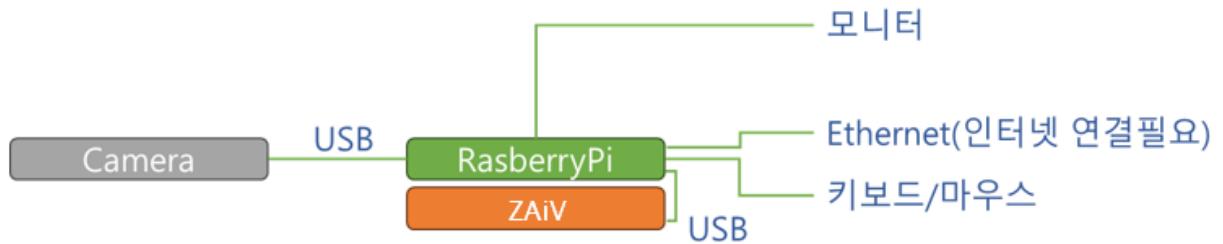
2. History

Version	날짜	내용
0.1	2023.06.28.	가이드 작성
0.2	2023.07.25	모델 재학습 작성
1.0	2023.08.08	ROS 환경 구성 및 실행 내용 작성
1.1	2023.08.09	오타 수정

II 실행 준비

1. 구성

1.1 환경 구성



[그림 II-1] 환경 구성

1.2 파일 준비

가이드와 함께 제공된 아래 파일리스트를 RaspberryPI의 작업 디렉토리로 복사합니다.

- 기본예제: cpp_yolov5_nms_eth.zip
- ROS 사용 예제: catkin_ws.zip
- Hailo 실행 라이브러리 및 파일: hailort_4.12.0_amd64.deb

2. 환경 구성

2.1 ZAiV IP 설정하기

2.1.1 네트워크 대역 설정

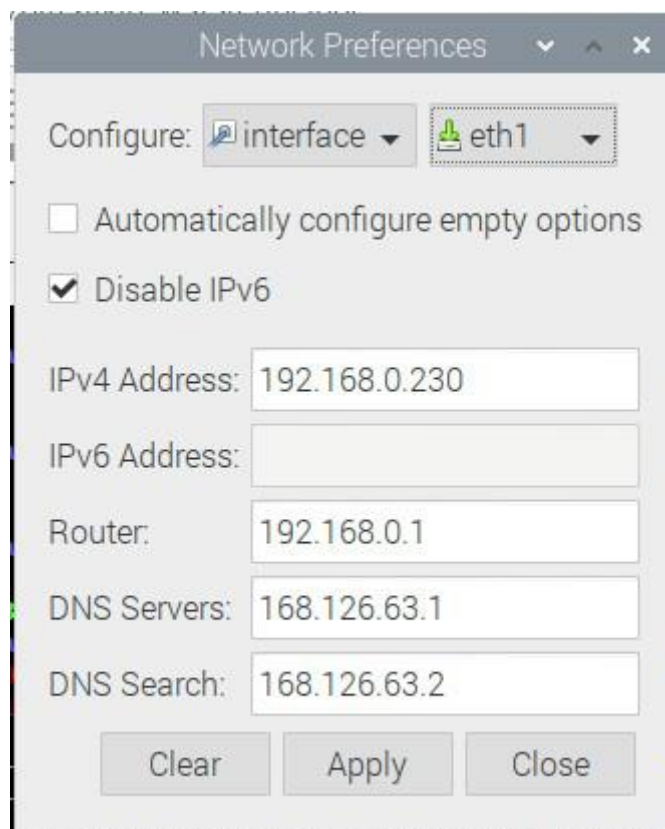
Board는 기본 네트워크 설정이 아래와 같이 설정되어 출하됩니다.

- IP: 192.168.0.11
- Gateway: 192.168.0.1
- Netmask: 255.255.255.0

ZAiV의 IP를 변경하고자 하는 경우, 먼저 RaspberryPI 및 Ubuntu의 네트워크 대역을 ZAiV와 동일한 대역으로 수정합니다.

예를 들어 ZAiV의 IP가 “192.168.0.11”인 경우, 1~254 중 Hailo가 사용중인 11을 제외한 숫자로 설정하고, Netmask를 “255.255.255.0”, Gateway를 “255.255.255.0”으로 설정합니다.

아래는 예시입니다.



[그림 II-2] Raspberry PI Network Interface

ZAiV User Guide

▪ 네트워크 인터페이스 확인

```
Ifconfig
```

아래의 경우 USB 버전 ZAiV보드가이므로 eth1이 hailo board와 연결된 인터페이스입니다.

만약 ethernet 버전 ZAiV를 사용하신다면 eth0을 hailo board와 네트워크 대역을 맞추어야 합니다.

```
pi@aMAP:~ $ ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.30.115 netmask 255.255.255.0 broadcast 192.168.30.255
    inet6 fe80::da3a:ddff:fe24:fa26 prefixlen 64 scopeid 0x20<link>
    ether d8:3a:dd:24:fa:26 txqueuelen 1000 (Ethernet)
    RX packets 341123 bytes 41070484 (39.1 MiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 277563 bytes 75028200 (71.5 MiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

eth1: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.0.230 netmask 255.255.255.0 broadcast 192.168.0.255
    inet6 fe80::7c35:e7ff:fe9c:8e83 prefixlen 64 scopeid 0x20<link>
    ether 7e:35:e7:9c:8e:83 txqueuelen 1000 (Ethernet)
    RX packets 19678 bytes 1379807 (1.3 MiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 19907 bytes 13178226 (12.5 MiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 9235 bytes 1735381 (1.6 MiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 9235 bytes 1735381 (1.6 MiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

wlan0: flags=4099<UP,BROADCAST,MULTICAST> mtu 1500
    ether d8:3a:dd:24:fa:28 txqueuelen 1000 (Ethernet)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
```

TX packets 0 bytes 0 (0.0 B)

TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

■ 설정된 인터페이스 동작 확인

해당 인터페이스로 scan을 하여 응답이 오고, 해당 IP로 Ping을 했을 때 응답이오면 정상 설정된 것입니다.

```
pi@aMAP:~ $ hailortcli scan --interface eth1
Hailo Ethernet Devices:
[-] Board IP: 192.168.0.11
pi@aMAP:~ $ ping 192.168.0.11
PING 192.168.0.11 (192.168.0.11) 56(84) bytes of data.
64 bytes from 192.168.0.11: icmp_seq=1 ttl=255 time=0.337 ms
64 bytes from 192.168.0.11: icmp_seq=2 ttl=255 time=0.285 ms
64 bytes from 192.168.0.11: icmp_seq=3 ttl=255 time=0.333 ms
64 bytes from 192.168.0.11: icmp_seq=4 ttl=255 time=0.302 ms
^C
--- 192.168.0.11 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 82ms
rtt min/avg/max/mdev = 0.285/0.314/0.337/0.025 ms
```

2.1.2 ZAiV 네트워크 대역 수정하기

ZAiV Board의 IP를 수정하고자 하는 경우, 아래 내용에 따라 수정가능합니다.

먼저, 아래 명령어를 통해 config.json파일을 읽어옵니다.

```
hailortcli fw-config read --output-file config.json --ip 192.168.0.11
```

Vi 편집기 등을 이용해 config.json파일을 열어옵니다.

```
{
  "network": {
    "should_use_dhcp": false,
    "static_ip_address": {
      "value": "192.168.0.11"      //고정 아이피 설정
    },
    "static_gw_address": {
      "value": "192.168.0.1"      //게이트웨이 설정
    },
    "static_netmask": {
      "value": "255.255.255.0"    //넷 마스크 설정
    },
    "rx_pause_frames_enable": true
  },
  "logger": {
    "send_via_pci": false,
    "send_via_uart": true
  }
}
```

사용하고자하는 네트워크 설정으로 수정하고,

아래 명령어를 통해 수정된 config.json을 기록합니다.

```
hailortcli fw-config write config.json --ip 192.168.0.11
```

ZAiV Edge 보드의 펌웨어 네트워크 설정을 변경한 경우, 동일하게 RaspberryPI의 Ethernet 인터페이스 네트워크 대역도 함께 변경해야 합니다.

2.2 종속성 설치

Withus의 기본 제공 이미지를 사용하시면 종속성 설치 항목은 건너뛰어도 됩니다.

2.2.1 기본 설치

터미널을 실행 후 먼저 필요한 종속성들을 설치합니다.

```
sudo apt-get install -y libopencv-dev
sudo apt-get install gcc-9 g++-9
sudo dpkg --install hailort_4.12.0_arm64.deb
```

※ OpenCV 설치 시 현재 커널 버전에 맞는 OpenCV가 설치되어 버전이 다를 수 있습니다.

“/압축해제 폴더/cpp_yolov5_nms_eth/CMakeLists.txt” 파일을 열어보면 “find_package(OpenCV 3.4 REQUIRED)” OpenCV 3.4 버전으로 작성되어 있어 만약 버전이 다른 경우에는 해당 부분에서 3.4를 지우시면 됩니다.

⇒ Line:20 find_package(OpenCV 3.4 REQUIRED) => find_package(OpenCV REQUIRED)

2.2.2 ROS (Melodic) 빌드 및 설치

ROS를 사용하는 경우에만 빌드 및 설치가 필요합니다.

▪ ROS Repositories 설치

```
sudo sh -c 'echo "deb http://packages.ros.org/ros/ubuntu $(lsb_release -sc) main" >
/etc/apt/sources.list.d/ros-latest.list'

sudo apt-key adv --keyserver 'hkp://keyserver.ubuntu.com:80' --recv-key
C1CF6E31E6BADE8868B172B4F42ED6FBAB17C654

sudo apt-get update

sudo apt-get install -y python-rosdep python-rosinstall-generator python-wstool python-rosinstall
build-essential cmake
```

▪ Bootstrap Dependencies 설치

```
sudo apt install -y python-rosdep python-rosinstall-generator python-wstool python-rosinstall build-essential cmake
```

▪ rosdep 초기화

```
sudo rosdep init
rosdep update
```

▪ catkin Workspace 생성

```
mkdir -p ~/ros_catkin_ws
cd ~/ros_catkin_ws
```

▪ Workspace 설치 (Desktop)

```
rosinstall_generator desktop --rostdistro melodic --deps --wet-only --tar > melodic-desktop-wet.rosinstall  
wstool init src melodic-desktop-wet.rosinstall
```

▪ Dependencies 설치

```
rosdep install -y --from-paths src --ignore-src --rostdistro melodic -r --os=debian:buster
```

▪ catkin Workspace 빌드

```
sudo ./src/catkin/bin/catkin_make_isolated --install -DCMAKE_BUILD_TYPE=Release --install-space /opt/ros/melodic
```

▪ source the new installation

```
source /opt/ros/melodic/setup.bash
```

▪ environment 변수 추가

```
echo "source /opt/ros/melodic/setup.bash" >> ~/.bashrc
```

▪ Released Packages 추가

```
rosinstall_generator ros_comm ros_control joystick_drivers --rostdistro melodic --deps --wet-only --tar > melodic-custom_ros.rosinstall  
rosinstall_generator compressed_image_transport --rostdistro melodic --deps --wet-only --tar > melodic-compressed_image_transport-wet.rosinstall  
rosinstall_generator camera_info_manager --rostdistro melodic --deps --wet-only --tar > melodic-camera_info_manager-wet.rosinstall  
rosinstall_generator dynamic_reconfigure --rostdistro melodic --deps --wet-only --tar > melodic-dynamic_reconfigure-wet.rosinstall  
  
wstool merge -t src melodic-custom_ros.rosinstall  
wstool merge -t src melodic-compressed_image_transport-wet.rosinstall  
wstool merge -t src melodic-camera_info_manager-wet.rosinstall  
wstool merge -t src melodic-dynamic_reconfigure-wet.rosinstall  
  
wstool update -t src  
rosdep install --from-paths src --ignore-src --rostdistro melodic -y -r --os=debian:buster  
sudo ./src/catkin/bin/catkin_make_isolated --install -DCMAKE_BUILD_TYPE=Release --install-space /opt/ros/melodic
```

3. 실행

3.1 기본 예제

3.1.1 압축해제

“cpp_yolov5_nms_eth.zip” 파일의 압축을 해제합니다.

```
unzip cpp_yolov5_nms_eth.zip
```

3.1.1.1 빌드

▪ 프로젝트 빌드

압축해제한 “cpp_yolov5_nms_eth.zip” 폴더로 들어가 빌드 스크립트를 통해 프로젝트를 빌드합니다.

```
cd cpp_yolov5_nms_eth.zip
./build.sh
```

▪ 결과 확인

빌드 완료된 실행 파일은 “[프로젝트경로]/build/aarch64/customer_demo” 파일입니다.

3.1.1.2 실행

실행 시 Ethernet interface의 이름 및 연결된 카메라 번호가 필요합니다.

```
./build/aarch64/customer_demo [Ethernet 인터페이스 이름] [카메라 번호]
ex) ./build/aarch64/customer_demo eth0 0
    ./build/aarch64/customer_demo eth1 3
```

▪ Ethernet 인터페이스 이름

Ethernet 인터페이스 이름은 ifconfig 명령을 통해 ZAiV Edge Board의 IP (Default: 192.168.0.11)와 대역대가 맞는 인터페이스 이름을 설정하시면 됩니다. 사용 중인 인터페이스의 IP 대역과 맞지 않아 ZAiV Edge Board의 IP를 변경하고자하는 경우 “설정”을 참조해주세요.

▪ 카메라 번호

카메라 번호는 /dev/video 번호를 가져오면 되며 v4l2-ctl --list-devices 명령어를 사용하여 확인할 수 있습니다.

```
pi@hailo:~ $ v4l2-ctl --list-devices
bcm2835-codec-decode (platform:bcm2835-codec):
    /dev/video10
    /dev/video11
    /dev/video12
    /dev/video18
    /dev/video31
    /dev/media2

bcm2835-isp (platform:bcm2835-isp):
    /dev/video13
    /dev/video14
    /dev/video15
    /dev/video16
    /dev/video20
    /dev/video21
    /dev/video22
    /dev/video23
    /dev/media1
    /dev/media3

rpiivid (platform:rpiivid):
    /dev/video19
    /dev/media0

HD USB Camera: HD USB Camera (usb-0000:01:00.0-1.2):
    /dev/video0
    /dev/video1
    /dev/media4
```

[그림 II-3] 카메라 번호

3.2 ROS 예제

3.2.1 준비

▪ ROS 워크스페이스 준비

```
mkdir -p ~/catkin_ws/src
cd ~/catkin_ws/src
catkin_init_workspace
```

▪ alias 등록

```
vi ~/.bashrc

# ~/.bashrc에 추가.
#####
alias cw='cd ~/catkin_ws && source ~/catkin_ws/devel/setup.bash'
alias cs='cd ~/catkin_ws/src && source ~/catkin_ws/devel/setup.bash'
alias cm='cd ~/catkin_ws && catkin_make && source ~/catkin_ws/devel/setup.bash'
```



```
alias vb='vim ~/.bashrc'
alias eb='gedit ~/.bashrc'
alias sb='source ~/.bashrc'

#####

source ~/.bashrc
```

▪ 소스 구성

```
cp hailo_ai.zip ~/catkin_ws/src
unzip ~/catkin_ws/src/hailo_ai.zip
```

압축해제 후 소스폴더의 구성

```
pi@aMAP:~/catkin_ws/src $ ll
total 48
lrwxrwxrwx 1 pi pi   50 Aug  8 17:43 CMakeLists.txt ->
/opt/ros/melodic/share/catkin/cmake/toplevel.cmake
drwxr-xr-x 7 pi pi 4096 Aug  8 20:37 hailo_ai
drwxr-xr-x 7 pi pi 4096 Aug  8 18:12 usb_cam
```

3.2.2 빌드

```
cd ~/catkin_ws
catkin_make
```

3.2.3 설정

3.2.3.1 Usb_cam 설정

- 카메라 지원 포맷 확인

```
pi@amAP:~/catkin_ws/src $ ffmpeg -f video4linux2 -list_formats all -i /dev/video0
ffmpeg version 4.1.11-0+deb10u1+rpt1 Copyright (c) 2000-2023 the FFmpeg developers
  built with gcc 8 (Debian 8.3.0-6)

  configuration: --prefix=/usr --extra-version=0+deb10u1+rpt1 --toolchain=hardened --
incdir=/usr/include/aarch64-linux-gnu --enable-gpl --disable-stripping --enable-avresample --
disable-filter=resample --enable-avisynth --enable-gnutls --enable-ladspa --enable-libaom --enable-
libass --enable-libbluray --enable-libbs2b --enable-libcaca --enable-libcdio --enable-libcodec2 --
enable-libflite --enable-libfontconfig --enable-libfreetype --enable-libfribidi --enable-libgme --
enable-libgsm --enable-libjack --enable-libmp3lame --enable-libmysofa --enable-libopenjpeg --enable-
libopenmpt --enable-libopus --enable-libpulse --enable-librsvg --enable-librubberband --enable-
libshine --enable-libsnappy --enable-libsoxr --enable-libspeex --enable-libssh --enable-libtheora --
enable-libtwolame --enable-libvidstab --enable-libvorbis --enable-libvpx --enable-libwavpack --
enable-libwebp --enable-libx265 --enable-libxml2 --enable-libxvid --enable-libzmq --enable-libzvbi -
-enable-lv2 --enable-omx --enable-opengl --enable-sdl2 --enable-omx-rpi --disable-
mmal --enable-neon --enable-vout-drm --enable-v4l2-request --enable-libudev --
libdir=/usr/lib/aarch64-linux-gnu --arch=arm64 --enable-libdc1394 --enable-libdrm --enable-
libiec61883 --enable-chromaprint --enable-frei0r --enable-libx264 --enable-shared

libavutil      56. 22.100 / 56. 22.100
libavcodec     58. 35.100 / 58. 35.100
libavformat    58. 20.100 / 58. 20.100
libavdevice    58.  5.100 / 58.  5.100
libavfilter     7. 40.101 /  7. 40.101
libavresample   4.  0.  0 /  4.  0.  0
libswscale      5.  3.100 /  5.  3.100
libswresample   3.  3.100 /  3.  3.100
libpostproc    55.  3.100 / 55.  3.100

[video4linux2,v4l2 @ 0x55817e5880] Raw          :      uyvy422 :      UYVY 4:2:2 : 640x480 960x540
1280x960 1280x720 1920x1080
```

- usb_cam.yml 파일 열기

```
Cd ~/catkin_ws/src
vi ./hailo_ai/usb_cam/config/usb_cam.yml
```

- Usb_cam.yml 파일 수정

주로 아래 항목들이 수정됩니다. 앞서 확인한 카메라 지원 정보에 따라 수정합니다.

- pixel_format
- color_format

- image_width
- image_height

3.2.3.2 ZAiV 설정

▪ Zaivrun.launch 파일 열기

```
Cd ~/catkin_ws/src  
vi ./hailo_ai/launch/zaivrun.launch
```

▪ Zaivrun.launch 파일 내용 수정

- CamNum: Camera의 Index number
- CamWidth: usb_cam 노드로부터 입력받을 가로해상도
- CamHeight: usb_cam 노드로부터 입력받을 세로해상도
- InterfaceName: ZAiV 보드가 연결된 Ethernet USB의 interface 명(ifconfig로 확인가능)
- HEF_File: ZAiV 보드 추론에 사용할 모델

```
<launch>  
  <param name="CamNum" value="0"/>  
  <param name="CamWidth" value="640"/>  
  <param name="CamHeight" value="480"/>  
  <param name="InterfaceName" value="eth1"/>  
  <param name="HEF_File" value="/home/pi/Work/hef_files/traffic_signal_230806.hef"/>  
  
  <!-- Publish diagnostics information from low-level MCU outputs -->  
  <node pkg="hailo_ai" name="hailo_ai_main" type="hailo_ai_node">  
  </node>  
  
  <!-- Publish wheel odometry from MCU encoder data -->  
  <node pkg="hailo_ai" name="hailo_ai_view" type="hailo_ai_view_node"/>  
</launch>
```

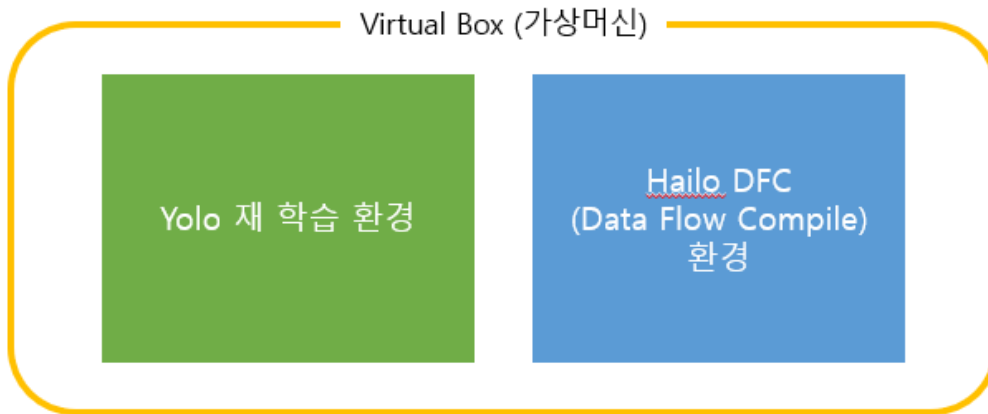
3.2.4 실행

```
# Node 실행
source ~/catkin_ws/devel/setup.bash
roslaunch usb_cam usb_cam.launch
roslaunch src/zaivrun.launch
```

III 모델 재학습

1. 구성

1.1 환경구성

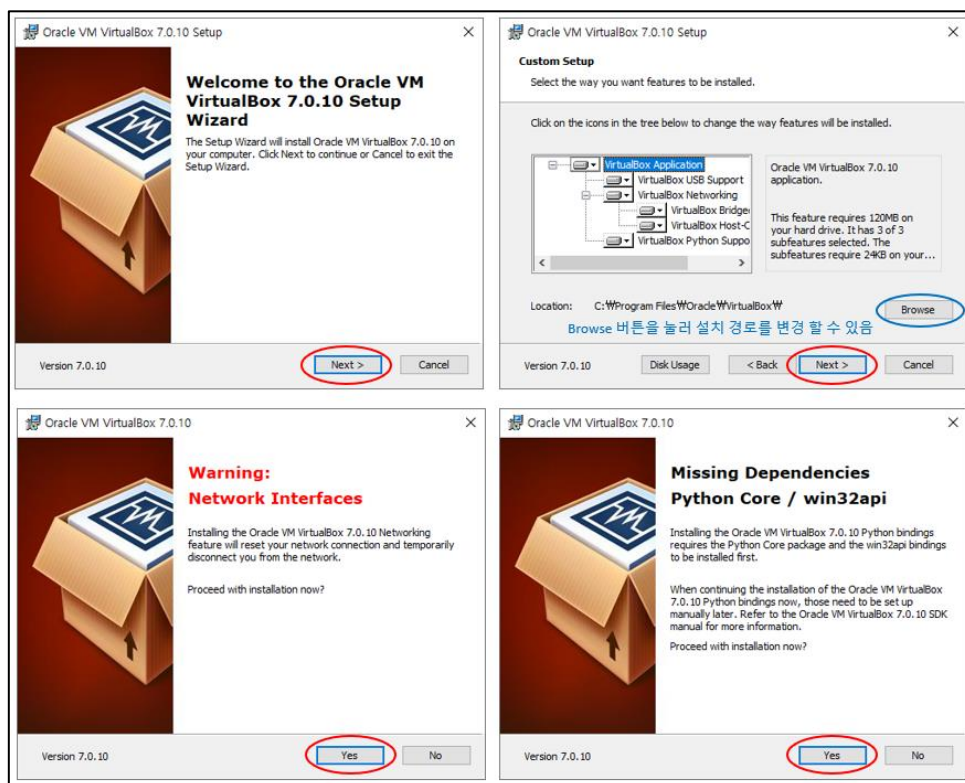


[그림 III-1] 환경구성

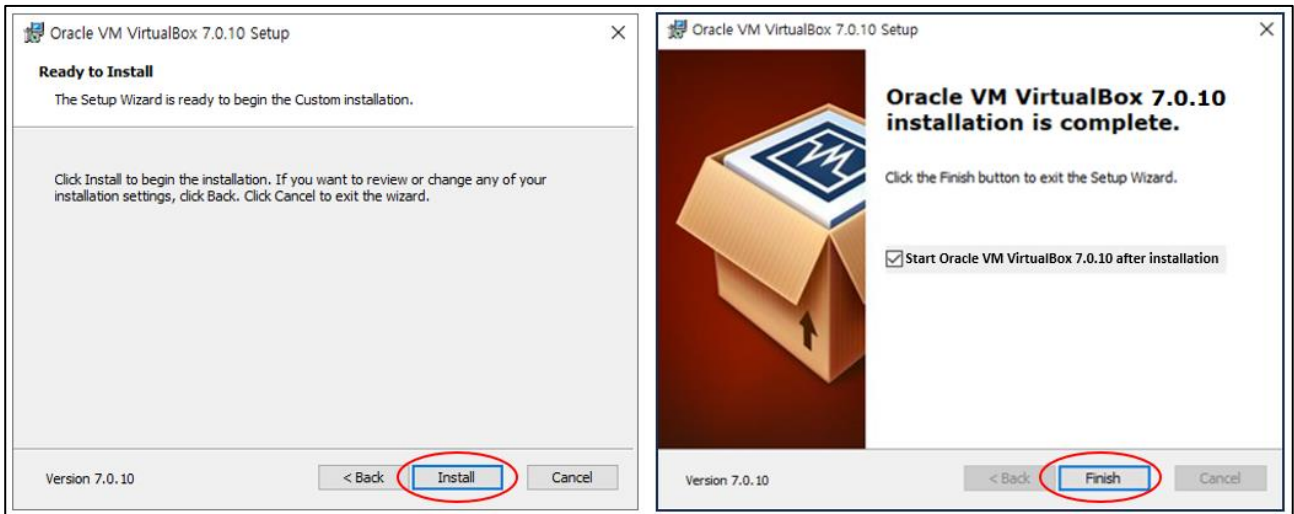
2. 가상머신

2.1 설치

가이드와 함께 제공된 VirtualBox-7.0.10-158379-Win.exe 설치파일을 이용하여 버추얼 박스를 설치합니다.



[그림 III-2] 설치 마법사 1



[그림 III-3] 설치 마법사 2

2.2 가상 환경 생성

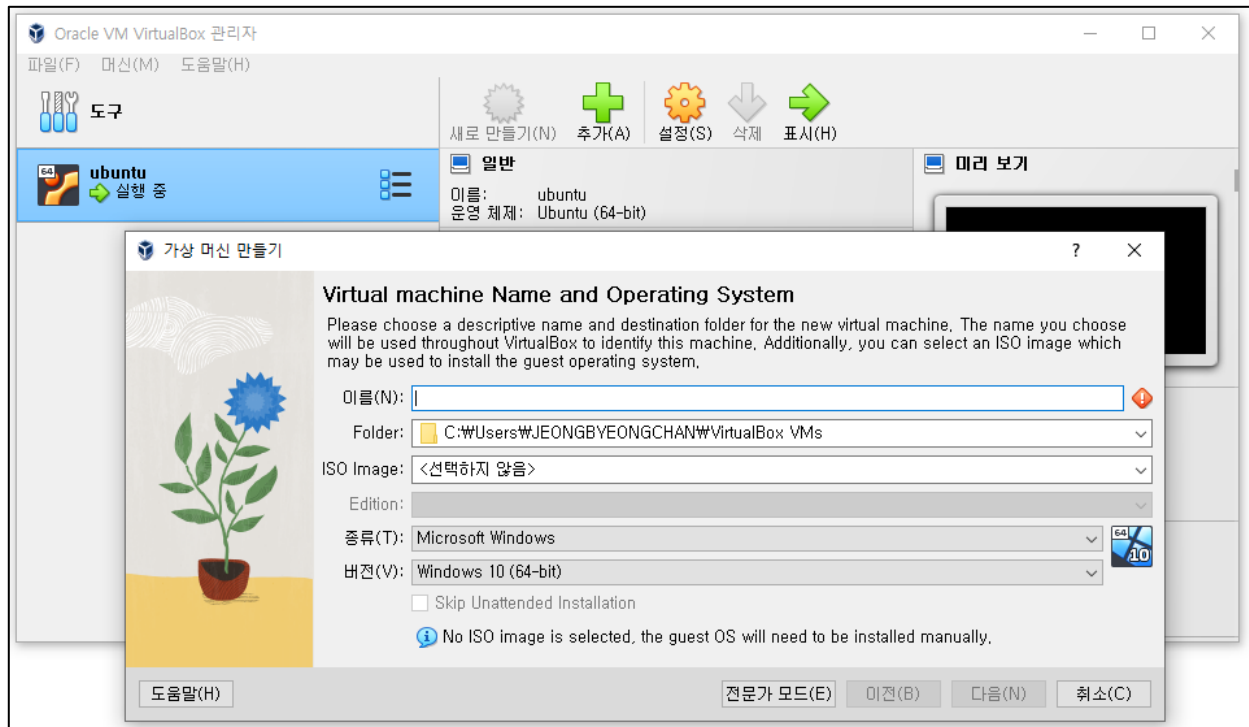
설치가 완료되면 아래와 같은 메인 화면이 출력됩니다.

메인 화면에서 yolov5 학습과 DataFlow Compiler를 위한 ubuntu 가상 환경을 만듭니다.



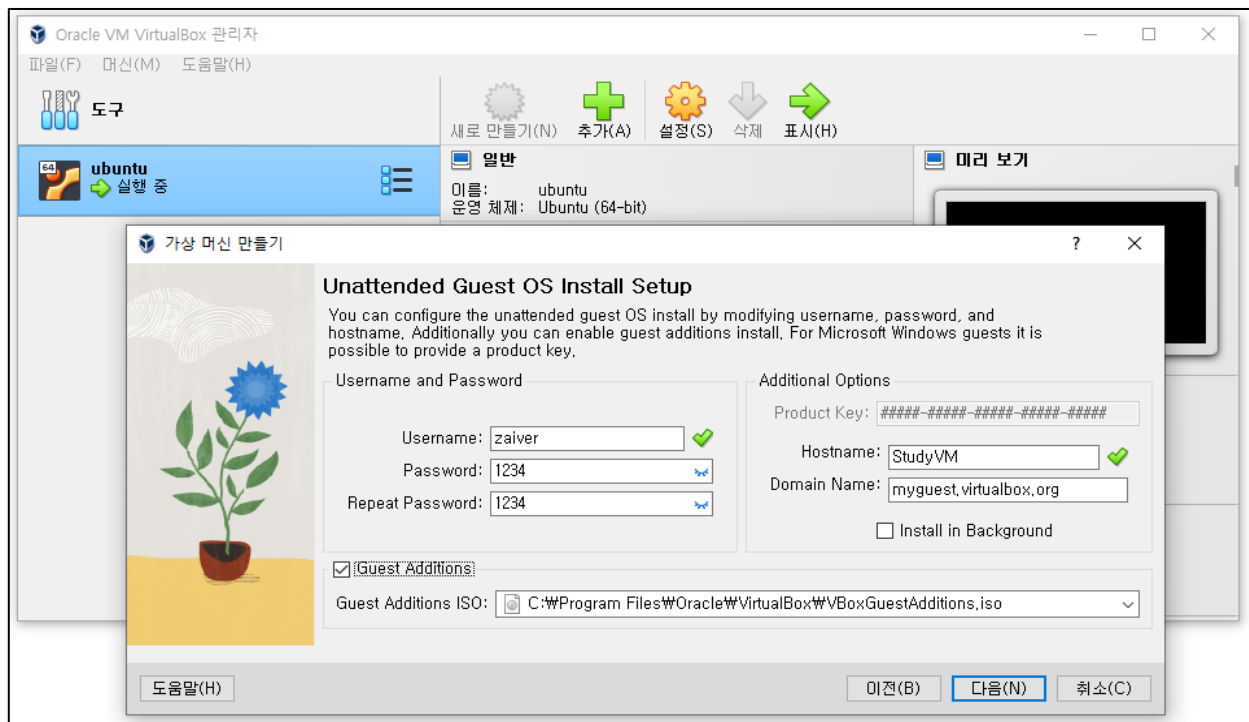
[그림 III-4] 가상 환경 생성 1

새로 만들기를 눌러 아래와 같은 창이 뜨면 (1)가상머신 이름을 지정, (2)저장할 경로를 지정, (3)부팅 디스크 파일인 **ubuntu-20.04.3-desktop-amd64.iso** 파일을 지정하여 다음 버튼을 클릭합니다.



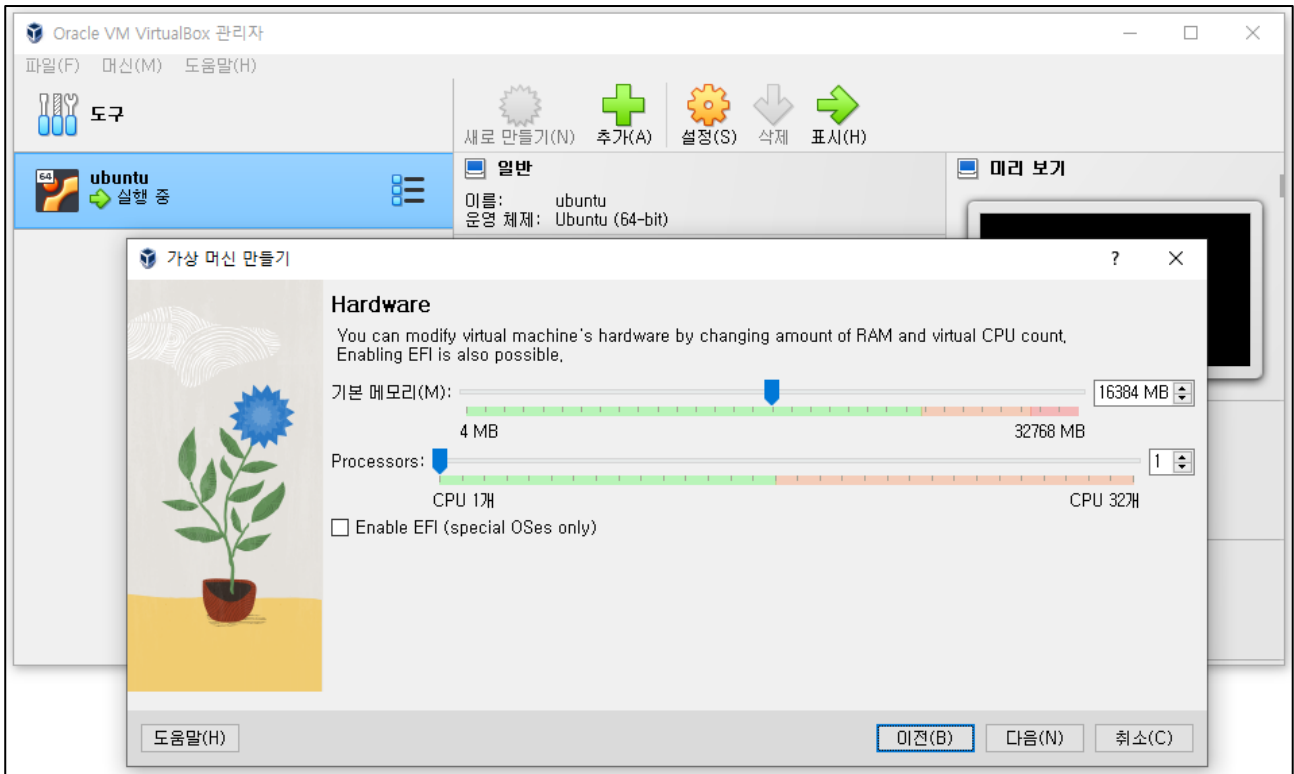
[그림 III-5] 가상 환경 생성 2

사용자 이름과 비밀번호, Hostname 을 지정하고 Guest Addition 를 체크하고 다음 버튼을 클릭합니다.



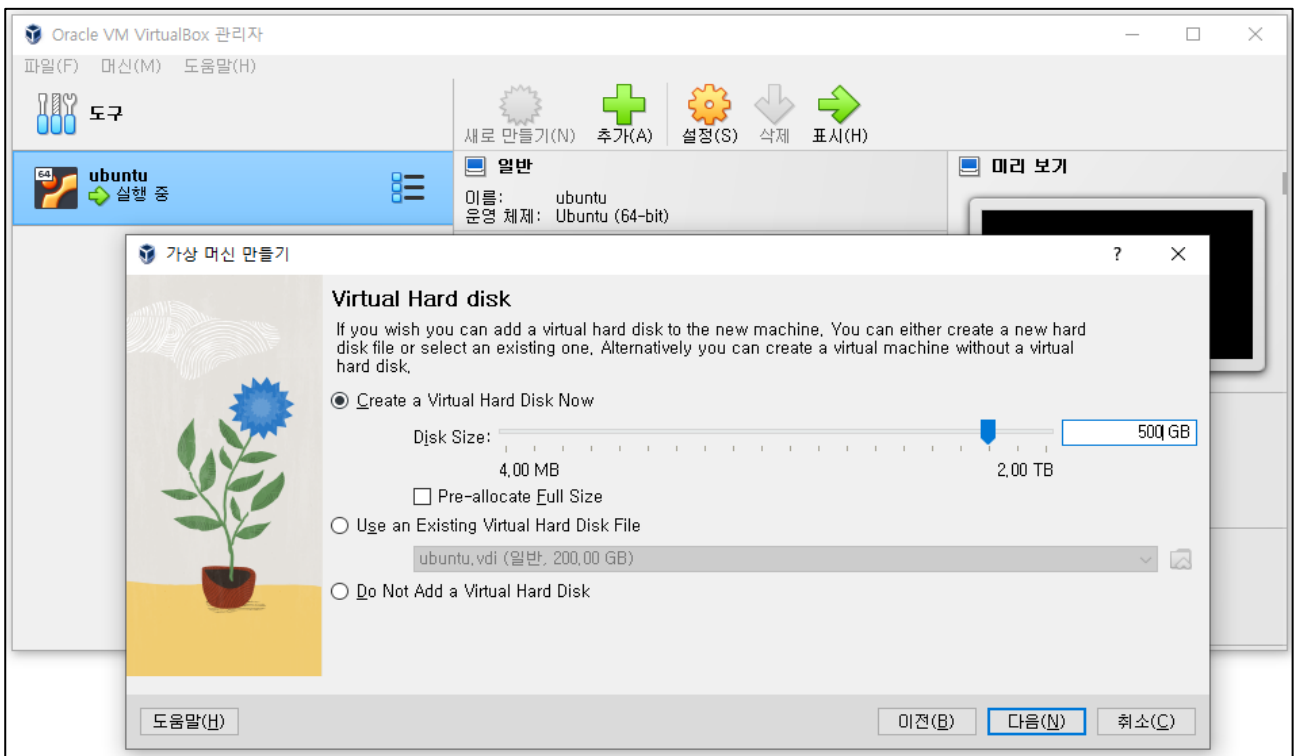
[그림 III-6] 가상 환경 생성 3

메모리는 10GB 이상으로 설정한 후 다음 버튼을 클릭합니다.



[그림 III-7] 가상 환경 생성 4

가상 하드디스크 용량을 500GB 정도로 설정하여 다음 버튼을 클릭합니다.

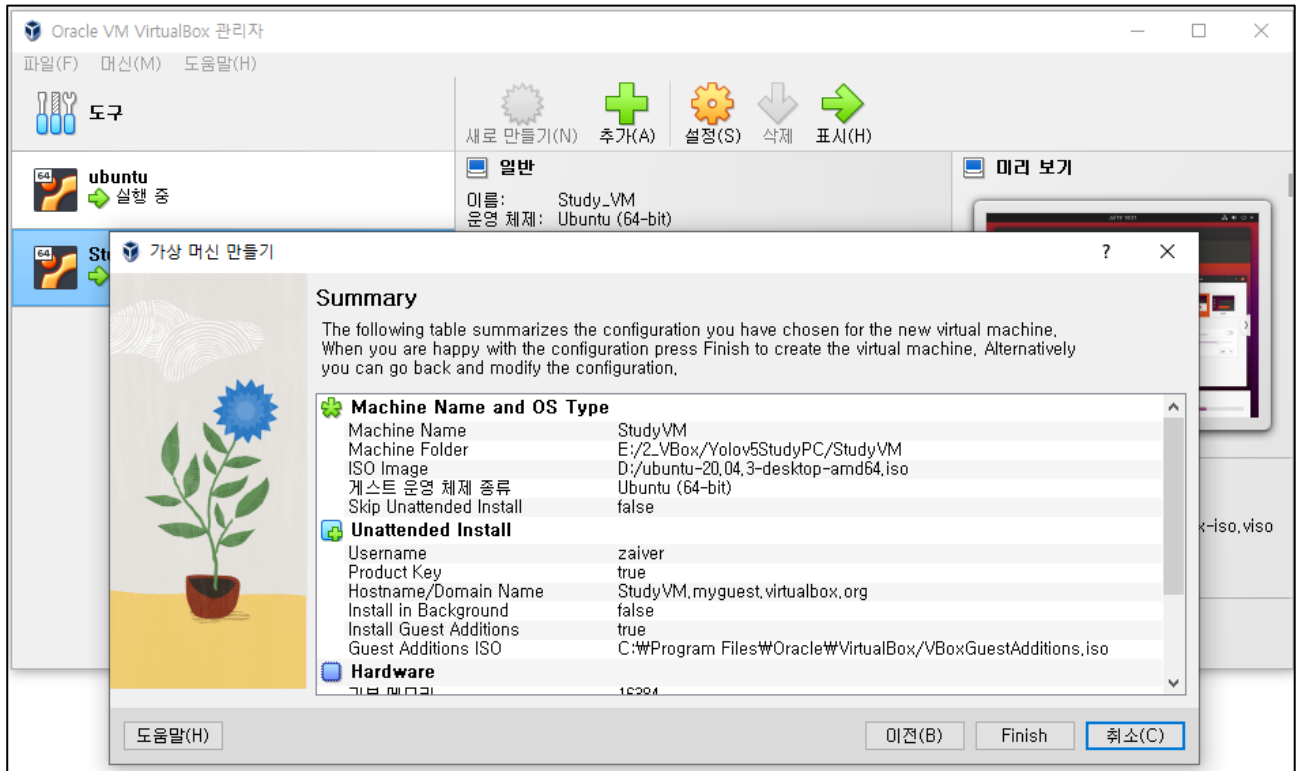


[그림 III-8] 가상 환경 생성 5

요약 창이 출력되면 Finish 눌러서 완료합니다.

완료된 가상머신은 자동으로 실행되며 설치를 진행하고 완료되는데까지 약 1 시간 정도 소요됩니다.

자동으로 실행되지 않는 경우 실행을 눌러 실행하면 됩니다.



[그림 III-9] 가상 환경 생성 6

3. YOLOv5 재학습 환경 구성

진행하면서 오류가 나는 경우 [예외](#) 섹션을 참조 바랍니다.

3.1 아나콘다 설치

실행된 가상환경에서 YOLOv5 모델을 커스텀 데이터셋으로 재학습하기 위해 필요한 파일을 구성합니다.

필요한 라이브러리 먼저 설치를 진행합니다.

```
sudo apt-get install Net-tools git libxcb-xinerama0
```

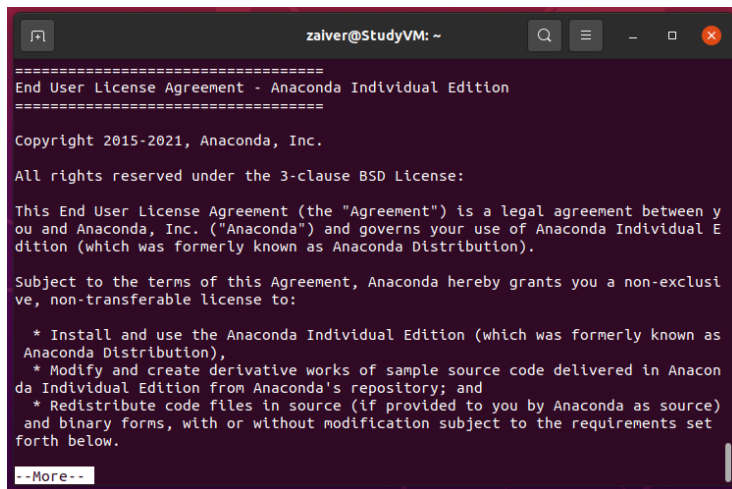
설치가 완료되면 가이드에 포함된 파일을 가상환경의 바탕화면으로 드래그 앤 드롭해서 옮깁니다.

- **Anaconda3-2021.05-Linux-x86_64.sh**
- **yolov5_retraining.ipynb**

두 파일을 가상머신에 옮기고 해당 경로 (~/Desktop\$) 에서 스크립트를 실행합니다.

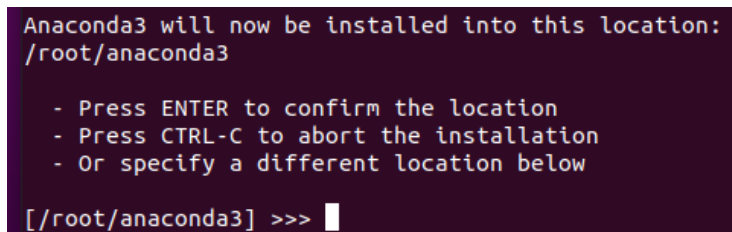
```
sudo bash Anaconda3-2021.05-Linux-x86_64.sh
```

- **아나콘다 설치 진행**



라이선스 확인은 q 를 눌러 넘기고
yes 를 입력합니다.

[그림 III-10] 라이선스 내용 확인



저장 경로를 지정합니다.
기본 경로는 /root/anaconda3 입니다.

[그림 III-11] 저장 경로 지정

```
Preparing transaction: done
Executing transaction: done
installation finished.
Do you wish the installer to initialize Anaconda3
by running conda init? [yes|no]
[no] >>>
```

Root 계정도 conda init 을 할것인지
물어보는 부분이며 yes / no
상관없습니다.

[그림 III-12] Root 계정 Conda init 유무

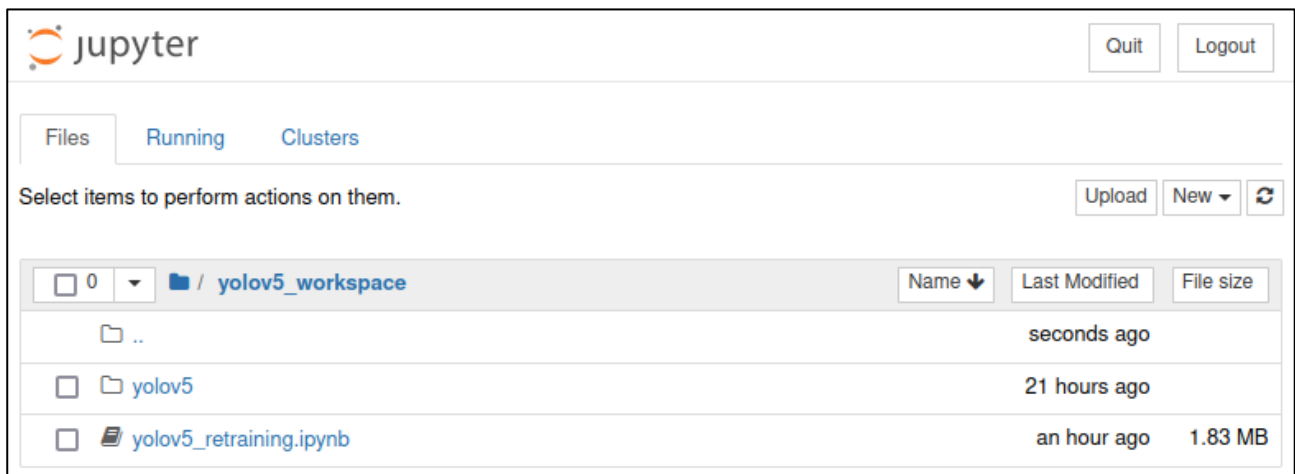
Conda 설치가 마무리되었으며 이제 설정을 진행합니다.

```
source /root/anaconda3/bin/activate -> conda 기본 경로이며 변경한 경우 변경된 path로 지정해야 합니다.
conda init
source ~/.bashrc
conda config --set auto_activate_base {True or False} conda 환경을 자동으로 활성화하는 옵션입니다.
conda create -n yolov5 python=3.9
conda activate yolov5
conda install jupyter
mkdir yolov5_workspace
```

우분투 화면에서 Yolov5_workspce 폴더를 열고 가이드에 포함된 yolov5_retraining.ipynb 파일과 data.yaml 파일을 드래그 앤 드롭으로 옮겨줍니다.

```
cd yolov5_workspace
Jupyter notebook
```

위 명령어를 실행하면 아래 같은 화면이 출력됩니다. yolov5_retraining.ipynb 파일만 있는게 정상입니다.



[그림 III-13] yolov5 Jupyter notebook

3.2 YOLOv5 설치 / 재학습

yolov5_retraining.ipynb 이라고 되어있는 노트북 파일을 열면 설명과 같이 yolov5 재학습 소스가 있습니다.

설명과 같이 확인하면서 진행하면 됩니다.

Yolov5 설치

yolov5 깃 서버에 접속하여 다운로드하고 Requirements.txt파일로 필요한 라이브러리들을 일괄 설치하는 과정입니다.

```
# clone YOLOv5 and
!git clone https://github.com/ultralytics/yolov5 # clone repo
%cd yolov5
!pip install -qr requirements.txt comet_ml tensorboard# install dependencies
```

[그림 III-14] git을 이용한 Yolov5 설치

이미지 파일 관리

이미지 파일을 관리하기 위한 폴더를 생성합니다.

- Train = 모델 학습에 사용될 학습용 데이터 셋이 저장될 공간
- Valid = 모델 학습에 사용될 검증용 데이터 셋이 저장될 공간
- Test = 모델 학습을 완료하고 모델 테스트를 위한 데이터 셋이 저장될 공간
- data.yaml = 학습할 때 사용할 데이터 정보 파일

```
!mkdir TrainDataset
!mkdir TrainDataset/train
!mkdir TrainDataset/train/images
!mkdir TrainDataset/train/labels

!mkdir TrainDataset/valid
!mkdir TrainDataset/valid/images
!mkdir TrainDataset/valid/labels

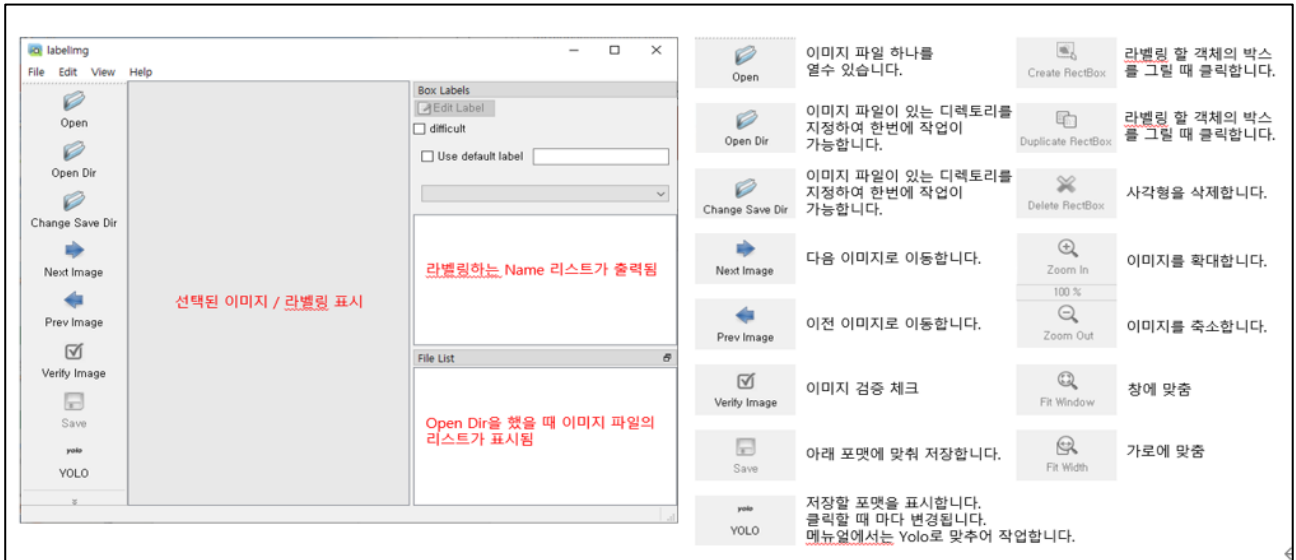
!mkdir TrainDataset/test
!mkdir TrainDataset/test/images
```

[그림 III-15] 데이터 셋 저장 공간

이미지 라벨링

무료 라벨링 프로그램인 labelImg를 사용하여 이미지 라벨링을 합니다.

!labelImg



[그림 III-16] 이미지 라벨링 툴

학습할 이미지 파일들을 이전에 생성한 **TrainDataset/train/images** 경로로 옮깁니다.

Open Dir 버튼을 눌러서 **TrainDataset/train/images** 경로로 지정하면 이미지들을 한번에 열고

Change Save Dir 버튼을 눌러서 **TrainDataset/train/labels** 경로로 지정합니다.

이미지를 열어 박스를 그리면 라벨이름을 지정하는 창이 출력되며 라벨이름은 저장되어 선택 리스트에 표시됩니다. - 라벨작업을 할 때 마다 저장합니다.

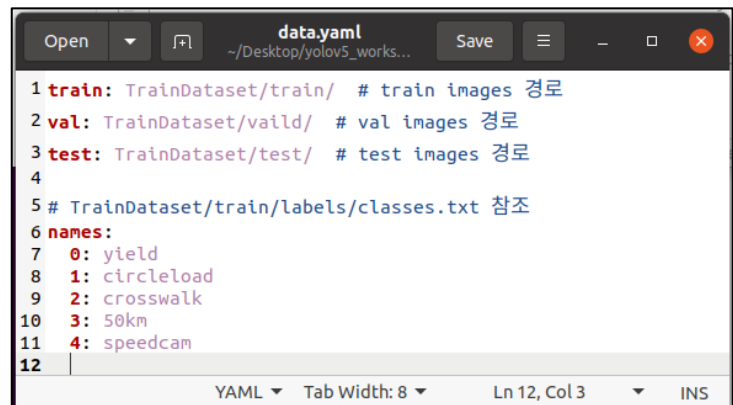
라벨링이 종료되면 train 데이터의 10%정도를 vaild 폴더로 옮겨줍니다.

TrainDataset/train/images 10% => TrainDataset/vaild/images
TrainDataset/train/labels 10% => TrainDataset/vaild/labels
TrainDataset/train/images 10% => TrainDataset/test/images

마지막으로 노트북 파일과 같이 옮겼던 data.yaml 파일을 열어서 수정합니다.

각각의 데이터 셋 경로를 지정해주고

라벨링툴을 이용해서 생성된 classes.txt 파일을 참조하여 순서대로 지정하면 됩니다.



이제 모델 학습을 시작하겠습니다.

Yolov5 모델 학습 시작

```
#필요 라이브러리 임포트하기
import torch
import os
from IPython.display import Image, clear_output # to display images

#모델 학습하기
!python train.py --img 640 --batch 4 --epochs 500 --data TrainDataset/data.yaml --weights yolov5n.pt
```

[그림 III-18] 모델 학습

Train.py 의 속성들은 img=이미지 사이즈, batch=학습 이미지 단위, epochs=학습 반복 횟수, data=데이터 셋 정보가 있는 yaml 파일 경로, weights=가중치 파일, cache=캐시메모리 사용 여부입니다.

```
#모델 학습하기
!python train.py --img 640 --batch 4 --epochs 500 --data TrainDataset/data.yaml --weights yolov5n.pt --cache

File "/home/zaiver/Desktop/yolov5_workspace/yolov5/utils/plots.py", line 291, in plot_images
    annotator.box_label(box, label, color=color)
File "/home/zaiver/Desktop/yolov5_workspace/yolov5/utils/plots.py", line 92, in box_label
    w, h = self.font.getsize(label) # text width, height (WARNING: deprecated in 9.2.0)
AttributeError: 'FreeTypeFont' object has no attribute 'getsize'
all      8      3      1      0.333      0.667      0.467
speedcam  8      3      1      0.333      0.667      0.467
Exception in thread Thread-20:
Traceback (most recent call last):
  File "/home/zaiver/.conda/envs/yolov5/lib/python3.9/threading.py", line 980, in _bootstrap_inner
    self.run()
  File "/home/zaiver/.conda/envs/yolov5/lib/python3.9/threading.py", line 917, in run
    self.target(*self.args, **self.kwargs)
  File "/home/zaiver/Desktop/yolov5_workspace/yolov5/utils/plots.py", line 291, in plot_images
    annotator.box_label(box, label, color=color)
  File "/home/zaiver/Desktop/yolov5_workspace/yolov5/utils/plots.py", line 92, in box_label
    w, h = self.font.getsize(label) # text width, height (WARNING: deprecated in 9.2.0)
AttributeError: 'FreeTypeFont' object has no attribute 'getsize'
Results saved to runs/train/exp2
```

[그림 III-19] 모델 학습 결과

학습이 완료되면 runs/train/exp 경로에 Loss 가 가장 적었던 best 모델(best.pt)이 저장됩니다.

```
# Start tensorboard
# Launch after you have started training
# logs save in the folder "runs"
%load_ext tensorboard
%tensorboard --logdir runs

!python detect.py --weights runs/train/exp/weights/best.pt --img 640 --conf 0.1 --source {dataset.location}/test/imag
```

[그림 III-20] 모델 결과 테스트

Tensorboard 를 이용하여 모델이 학습된 세부 내용들을 확인할 수 있고 detect.py 를 이용하여 학습된 모델로 추론 결과를 테스트할 수 있습니다.

```
python export.py --opset 15 --weights './runs/train/exp/weights/best.pt' --include torchscript onnx

export: data=data/coco128.yaml, weights=['./runs/train/exp/weights/best.pt'], imgsz=[640, 640], batch size=1, device
=cpu, half=False, inplace=False, keras=False, optimize=False, int8=False, dynamic=False, simplify=False, opset=15, v
erbose=False, workspace=4, nms=False, agnostic_nms=False, topk_per_class=100, topk_all=100, iou_thres=0.45, conf_thr
es=0.25, include=['torchscript', 'onnx']
YOLOv5 🚀 v7.0-53-g65071da Python-3.8.16 torch-1.13.0+cu116 CPU

Fusing layers...
Model summary: 157 layers, 7029004 parameters, 0 gradients, 15.8 GFLOPs

PyTorch: starting from runs/train/exp/weights/best.pt with output shape (1, 25200, 12) (13.7 MB)

TorchScript: starting export with torch 1.13.0+cu116...
TorchScript: export success ✅ 2.2s, saved as runs/train/exp/weights/best.torchscript (27.3 MB)

ONNX: starting export with onnx 1.13.0...
ONNX: export success ✅ 1.0s, saved as runs/train/exp/weights/best.onnx (27.2 MB)

Export complete (3.9s)
Results saved to /content/yolov5/runs/train/exp/weights
Detect:      python detect.py --weights runs/train/exp/weights/best.onnx
Validate:    python val.py --weights runs/train/exp/weights/best.onnx
PyTorch Hub: model = torch.hub.load('ultralytics/yolov5', 'custom', 'runs/train/exp/weights/best.onnx')
Visualize:   https://netron.app
```

[그림 III-21] yolov5 모델을 onnx로 내보내기

Export.py 를 이용해 onnx 파일로 저장해주면 커스텀 데이터 셋으로 모델 학습이 완료되었습니다.

4. DFC 환경 구성

DFC는 Dataflow Compiler의 약자로 AI 모델을 Hailo에서 인식/동작 할 수 있도록 양자화하고 컨버팅 하는 컴파일러입니다.

4.1 DFC 설치

시스템 요구사항은 아래와 같습니다.

- Ubuntu 20.04/22.04, 64-bit (supported also on Windows, under WSL2)
- 16+ GB RAM (32+ GB recommended)
- Python 3.8/3.9/3.10, including pip and virtualenv
- python3.X-dev and python3.X-distutils (according to the Python version), python3-tk, graphviz, and libgraphviz-dev packages. Use the command `sudo apt-get install PACKAGE` for installation.

지원가능 한 GPU 스펙은 아래와 같습니다. (권장사양. 가상환경에서는 GPU를 쓰지 않습니다.)

- Nvidia's Pascal/Turing/Ampere GPU architecture (such as Titan X Pascal, GTX 1080 Ti, RTX 2080 Ti, or RTX A4000)
- GPU driver version 470
- CUDA 11.2
- CUDNN 8.1

DFC설치를 위해서 필요한 패키지 들을 먼저 설치합니다.

```
sudo apt-get install python3.9-dev python3.9-distutils python3-tk python3-pip graphviz libgraphviz-dev python3-virtualenv
```

그리고 virtualenv를 이용하여 DFC 환경을 구성합니다.

```
virtualenv dfc_workspace
```

명령어를 실행하면 dfc_workspace라는 가상환경을 구성할 폴더가 생깁니다.

해당 폴더의 bin/activate를 실행하여 가상환경을 켜고 deactivate 명령어로 종료할 수 있습니다.

```
. dfc_workspace/bin/activate
```

가이드와 함께 제공된 hailo_dataflow_compiler-3.24.0-py3-none-linux_x86_64.whl 파일을 dfc_workspace 폴더 안에 복사하고 아래 명령어로 설치를 진행합니다.

```
pip install dfc_workspace/hailo_dataflow_compiler-3.24.0-py3-none-linux_x86_64.whl
```

설치가 완료되면 hailo -h를 입력해 결과가 잘 나오는지 확인합니다.


```
positional arguments:
  {analyze-noise,compiler,params-csv,parser,profiler,optimize,tb,visualizer,tutorial,har,join,har-onnx-rt,help}
  analyze-noise      Hailo utilities aimed to help with everything you need
  compiler           Analyze network quantization noise
  params-csv         Compile network to HEF binary files
  parser             Convert translated params to csv
  profiler           Translate network to Hailo network
  optimize           Hailo models Profiler
  tb                Optimize model
  visualizer         Convert *.har or *.ckpt to Tensorboard
  tutorial           HAR visualization tool
  har               Runs the tutorials in jupyter notebook
  join              Query and extract information from Hailo Archive file
  har-onnx-rt        Join two models to a single model
  help              Generates ONNX-Runtime model including pre/post
                   processing
                   Show the list of commands

optional arguments:
  -h, --help          show this help message and exit
  --version            show program's version number and exit
(df_c_workspace) zaiver@StudyVM:~/Desktop/dfc_workspace$ hailo -h
```

[그림 III-22] hailo -h 결과

```
cd /home/zaiver/Desktop/dfc_workspace/lib/python3.8/site-packages/hailo_tutorials/notebooks
mkdir ZAiV
cd ZAiV
nautilus .
```

위 명령어를 실행하면 ZAiV라는 이름으로 빈 폴더가 생성된 후 화면에 열립니다.

해당 폴더에 가이드와 함께 제공된 Yolo_dfc.zip 파일을 옮기고 풀어줍니다.

```
unzip yolo_dfc.zip
```

■ 파일 목록

- YOLO_Onnx_sign.ipynb (Yolo_dfc 노트북 파일)
- yolov5n_sign_20230718_v1.onnx (yolo model onnx 파일)
- sign_img (Cal 이미지 파일경로)
- nms_config_yolov5_sign.json (yolo_nms 설정 파일)

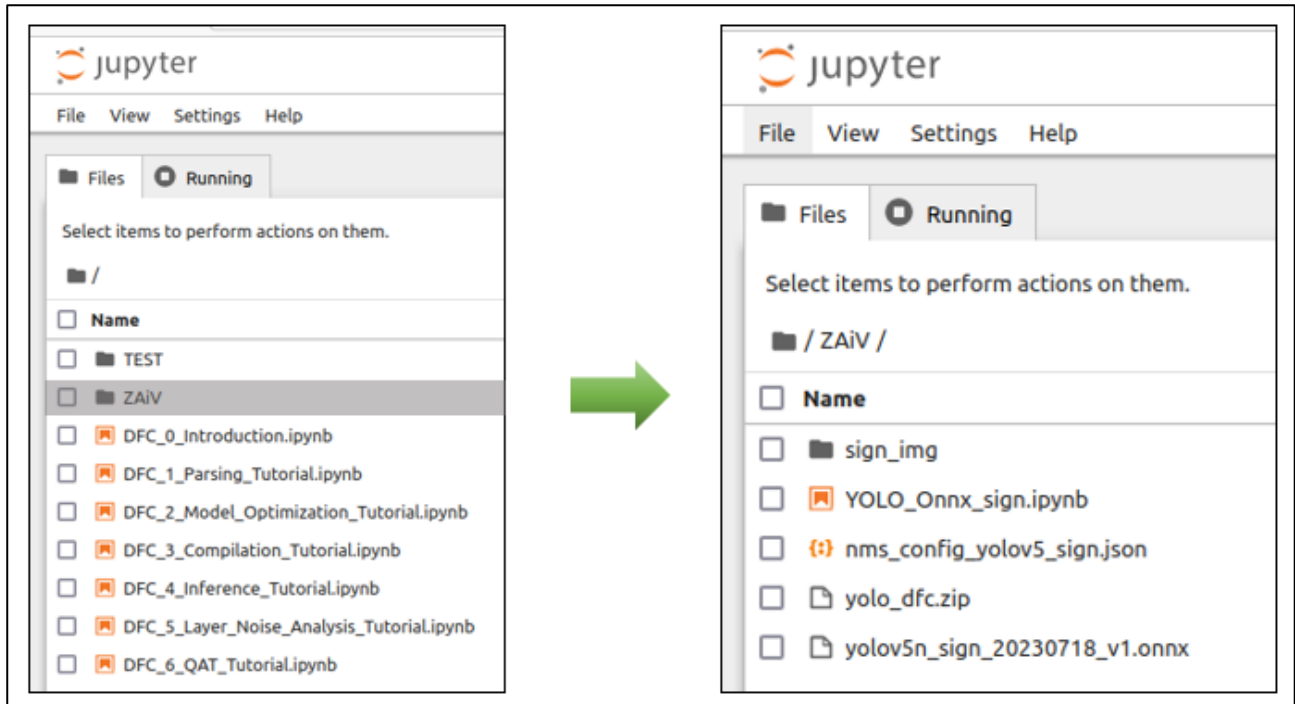
노트북 파일을 실행하기위해 명령어를 실행합니다.

```
hailo tutorial
```

그러면 주피터 노트북 환경이 실행되며 기본 튜토리얼인 7개 파일을 볼 수 있습니다.

ZAiV 폴더를 더블클릭하여 이동하면 아까 압축해제한 폴더를 볼 수 있습니다.

4.2 Data Flow Compiler 실행하기



[그림 III-23] ZAiV 폴더 이동

이전에 학습시킨 Custom 데이터 셋 Yolov5 모델 (.onnx)을 가지고 DFC 튜토리얼 내용을 참고하여 만든 파일이 YOLO_Onnx_sign 입니다. 표지판 데이터를 가지고 학습한 표지판 감지 모델입니다.

▪ 파일 구조

- Sign_img : 최적화에 사용되는 캘리브레이션 데이터 셋 (train 이미지 또는 test이미지 데이터 셋에서 일부 가져오면 됨 권장 사양은 1024장이나 가이드는 간소화하여 40장 미만 의 데이터 로 진행)
- YOLO_Onnx_sign.ipynb : 가이드 용 노트북 파일입니다.
- Nms_config_yolov5_sign.json : AI 가속기 칩(hailo) 에서 NMS라는 알고리즘을 이용한 후처리를 진행 할 수 있도록 설정파일을 넣어줌
- Yolov5n_sign_20230718_v1.onnx : Yolov5 재학습 결과 파일

가이드에서는 YOLO_Onnx_sign.ipynb 파일로 설명 하겠습니다.

첫번째로 파싱을 진행합니다. Onnx 모델을 AI 가속기가 인식하기 위해서 해주는 작업입니다.

YOLO Compile from ONNX

Parsing

```

%matplotlib inline
import numpy as np
import tensorflow as tf
import os
from IPython.display import display, SVG
from matplotlib import pyplot as plt
from PIL import Image

from hailo_sdk_client import ClientRunner
from hailo_sdk_common.targets.inference_targets import ParamsKinds

# Parse SSD model
yolo_model_name = 'sign_230718'
yolo_onnx_model_path = 'yolov5n_sign_20230718_v1.onnx'
start_node = ['images']
# end_nodes = ["Conv_198", "Conv_233", "Conv_268"] large, medium, small
# end_nodes = ["Conv_218", "Conv_235", "Conv_252"]
end_nodes = ["/model.24/m.0/Conv", "/model.24/m.1/Conv", "/model.24/m.2/Conv"]
runner1 = ClientRunner(hw_arch='hailo8')
hn, npz = runner1.translate_onnx_model(model=yolo_onnx_model_path,
                                     net_name=yolo_model_name,
                                     start_node_names=start_node,
                                     end_node_names=end_nodes,
                                     net_input_shapes={'images':[1, 3, 640,640]})

[info] NMS structure of yolov5 (or equivalent architecture) was detected, a generated config file will be saved inside the har. To add NMS postprocessing to the model, please use the command: nms_postprocess(meta_arch=yolov5).
[info] Translation completed on ONNX model sign_230718
[info] Initialized runner for sign_230718

```

모델 이름을 지정하고 onnx 파일의 경로와 Start Node, End Node 를 지정하여

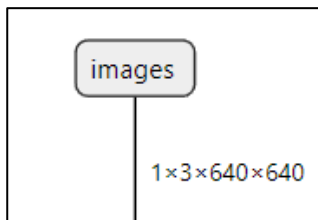
[그림 III-24] onnx 모델 파싱

Translate_onnx_model 함수에 넣어주면 hn, npz 의 hailo 형식으로 변환이 완료됩니다.

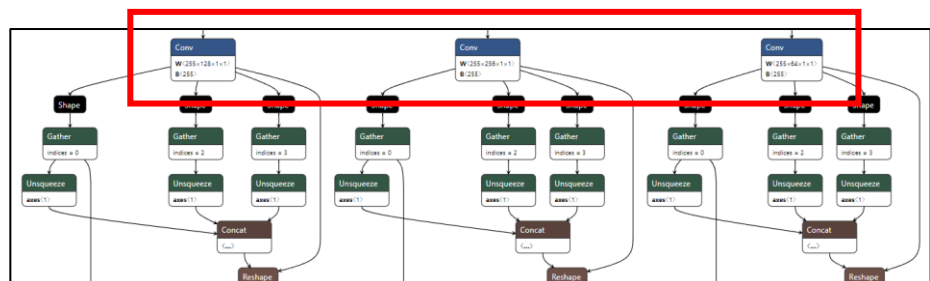
Start Node 와 End Node 를 찾는 방법은 Netron 을 이용하는 방법이 있습니다.

가상환경의 파이어폭스로 Netron 을 검색하면 웹으로 이용이 가능하며 open Model 버튼을 눌러서 시각화 할 onnx 파일을 지정해주면 시각화해서 보여줍니다.

(가이드의 모델 경로는 /home/zaiver/Desktop/dfc_workspace/lib/python3.8/site-packages/hailo_tutorials/notebooks/ZAiV/yolov5n_sign_20230718_v1.onnx 입니다.)



[그림 III-25] Start Node



[그림 III-26] EndNode

End node는 클릭해서 Node Properties에서 name 속성을 가져오면 됩니다.

파싱을 완료한 상태를 저장하고 visualizer 를 하면 변환된 모델을 netron 과 비슷하게 시각화 할 수 있습니다. 파싱하면서 이름들이 재정의되므로 SVG 파일을 한번 보면 이후에 nms 부분을 이해하는데 도움이 됩니다.

```
har_name = '{}_parse.har'.format(runner1.model_name)
runner1.save_har(har_name)
!hailo visualizer {har_name} --no-browser
SVG('sign_230718.svg')
```

[그림 III-26] 파싱 결과 저장 / 시각화

아래 내용의 주석을 풀면 파싱된 모델의 정보가 있는 html 파일이 열립니다.

```
: #har_name = '{}_parse.har'.format(runner1.model_name)
#!hailo profiler {har_name+'_parse'}
```

[그림 III-27] 파싱 결과 profiler 코드

다음은 파싱한 모델에 캘리브레이션 데이터 셋과 모델 스크립트(.alls)를 이용하여 optimize를 합니다. 캘리브레이션 데이터셋(calib_dataset)은 읽어오는 방법이 있고 생성하는 방법이 있습니다

```
#load calib data
# calib_dataset = np.load('calib_set_rgb.npy')
# calib_dataset = np.load('calib_set_sign.npy')
```

[그림 III-28] 캘리브레이션 데이터 읽어오기

생성하는 방법은 아래와 같습니다.

1. 이미지 전처리

```
from tensorflow.python.eager.context import eager_mode

def preproc(image, output_height=640, output_width=640, resize_side=640):
    ''' imagenet-standard: aspect-preserving resize to 256px smaller-side, then central-crop to 224px'''
    with eager_mode():
        h, w = image.shape[0], image.shape[1]
        scale = tf.cond(tf.less(h, w), lambda: resize_side / h, lambda: resize_side / w)
        resized_image = tf.compat.v1.image.resize_bilinear(tf.expand_dims(image, 0), [int(h*scale), int(w*scale)])
        cropped_image = tf.compat.v1.image.resize_with_crop_or_pad(resized_image, output_height, output_width)
        return tf.squeeze(cropped_image)
```

[그림 III-29] 이미지 전처리 함수 생성

2. 이미지 로드 / 전처리

```
images_path = 'sign_img/'
images_list = [img_name for img_name in os.listdir(images_path) if
                os.path.splitext(img_name)[1] == '.jpg']
calib_dataset = np.zeros((len(images_list), 640, 640, 3), dtype=np.float32)

for idx, img_name in enumerate(sorted(images_list)):
    img = np.array(Image.open(os.path.join(images_path, img_name)))
    img_preproc = preproc(img)
    calib_dataset[idx,:,:,:] = img_preproc.numpy().astype(np.uint8)
```

[그림 III-30] 이미지 불러오기 & 전처리

3. 데이터 셋 저장 및 확인

```
np.save('calib_set_sign.npy', calib_dataset)
plt.imshow(img, interpolation='nearest')
plt.title('Original image')
plt.show()
plt.imshow(np.array(calib_dataset[idx,:,:,:], np.uint16), interpolation='nearest')
plt.title('Preprocessed image')
plt.show()
```

[그림 III-31] 데이터셋 저장 & 확인하기

캘리브레이션 데이터셋 생성이 완료되면 모델 스크립트 파일을 작성합니다.

```
alls_lines_yolo= [
    'normalization1 = normalization([0.0, 0.0, 0.0], [255.0, 255.0, 255.0])\n',
    # 'nms_postprocess(meta_arch=yolo, image_dims=[512, 512], classes=6)\n',
    'nms_postprocess("./nms_config_yolov5_sign.json",yolov5)\n',
    'change_output_activation(sigmoid)\n',
    'model_optimization_config(calibration,batch_size=4,calibset_size=2)\n',
    # 'post_quantization_optimization(bias_correction, policy=enabled)\n',
    'post_quantization_optimization(bias_correction, policy=disabled)\n',
    'post_quantization_optimization(finetune,policy=disabled)\n',
    'compilation_param({conv*}, balance_output_multisplit=False)\n',
    # 'compilation_param(conv1, balance_output_multisplit=True)\n',
    'allocator_param(automatic_ddr=False)\n',
    'platform_param(targets=[ethernet])\n',
    'resources_param(strategy=greedy, max_control_utilization=0.9, max_compute_utilization=0.9, max_memory_utilization=0.9)\n',
    'context_switch_param(mode=disabled)\n',
    'performance_param(fps=240)'
]

# Save the commands in an .alls file, this is the Model Script
open('yolo_script.alls','w').writelines(alls_lines_yolo)

# Load the model script to ClientRunner so it will be considered on optimization
runner1.load_model_script('yolo_script.alls')

#runner1.optimize(calib_set)
runner1.optimize(calib_dataset)
#hn_layers = runner1.get_hn_dict()['layers']
#print([layer for layer in hn_layers if hn_layers[layer]['type'] == 'input_layer']) # See available input layer names
```

[그림 III-32] 모델 스크립트 및 최적화

모델 스크립트의 내용은 현재 ZAiV Ethernet 버전 전용으로 작성 되어있습니다.

수정할 필요가 있는 내용은 nms_postprocess 라인의 nms_config_yolov5_sign.json 파일과 model_optimization_config 라인의 batch_size, calibset_size 값 입니다.

Nms_config_yolov5_sign.json 내용 - ✓표시는 중요도입니다.

- Nms_scores_th : nms 에서 사용되는 임계값 ✓✓
- nms_iou_th : iou를 계산할 임계값 ✓✓
- image_dims : 입력 shape ✓
- classes : 클래스 개수 (10개 이하) ✓✓✓
- bbox_decoders → encoded_layer : 파싱한 후의 EndNode name을 입력 ✓

현재 가이드에 포함된 모델은 **classes** 개수가 4 개이며 클래스 개수가 다른 경우에는 수정이 필요합니다.

NMS 알고리즘에 적용될 임계값 (nms_scores, nms_iou) 들은 성능 평가에 영향을 줍니다.

(필요한 경우 변경해보며 테스트합니다.)

Image_dims 는 입력 shape 사이즈 이므로 맞춰주면 됩니다.

Bbox_decoders 의 encoded_layer 는 파싱 후 visualizer 를 실행하여 확인할 수 있는 .svg 파일에서의 output layey 이름입니다. (작성된 기준 모델은 yolov5n.pt 이며 모델이 다르면 다를 수 있습니다.)

model_optimization_config 내용을 수정하면 **calibration** 과정이 변경됩니다.

가이드는 매우 간소화 되어있으며 기본값은 **batch_size=2, calibset_size=64** 입니다.

위 과정을 거쳐 optimize 가 완료되면 저장하고 visualizer 를 하면 이전과 어떻게 달라졌는지 시각적으로 확인 가능하며 profiler 로 자세한 내용을 확인할 수 있습니다.

```
har_name = '{}_quantized.har'.format(runner1.model_name)

runner1.save_har(har_name)
!hailo visualizer {har_name} --no-browser

!hailo profiler {har_name}
```

[그림 III-33] 최적화 결과 확인

마지막으로 Comfile을 하면 라즈베리에서 사용가능한hef 파일이 생성됩니다.

```
Compilation

In this step we will compile the network to Hailo-8 binary files (HEF).

hef = runner1.compile()

file_name = '{}.hef'.format(runner1.model_name)
with open(file_name, 'wb') as f:
    f.write(hef)
```

[그림 III-34] hef 컴파일 코드

가이드로 제공된 ipynb 파일은 Run->Run All Cells 를 눌러 전체 실행이 가능합니다.

5. 기타사항

5.1 예외

- Sudo 권한 없음 설정 필요
 - <https://www.cyberithub.com/solved-user-is-not-in-the-sudoers-file-this-incident-reported>

5.2 Troubleshooting

- 터미널 안 열림
 - <https://code-lab1.tistory.com/309>